

Patient Management System

Project Guide for Collaborative Development, Code Structure, and Workflow Analysis

Project Overview & Learning Goals

This document provides a comprehensive, interactive view of the Patient Management System—a robust **Python-based application** for managing patient appointments. It emphasizes error prevention (e.g., double-booking), reporting, and modular design to foster practical software engineering skills.

Key Features

- **Patient Registration:** Unique patient profiles with ID, name, age, and medical condition.
- **Appointment Scheduling:** Define slots with doctor, date, time, and flexible capacity limits.
- **Intelligent Assignment:** Validate assignments to avoid overbooking or duplicates using custom exceptions.
- **Advanced Reporting:** Generate per-patient or per-appointment summaries for better oversight.
- **Modular Architecture:** Promotes code reusability and maintainability through OOP principles.

Collaborative Task Breakdown

Student A: Implement Patient and Appointment Classes (Entity Layer).

Student B: Develop AppointmentManager and Custom Exceptions (Business Logic Layer).

Student C: Create Reporting Functions and Main Menu Interface (Presentation Layer).

Learning Outcomes

Core Skills: OOP, Data Structures (dicts/lists), Exception Handling.

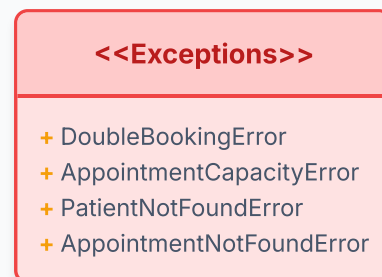
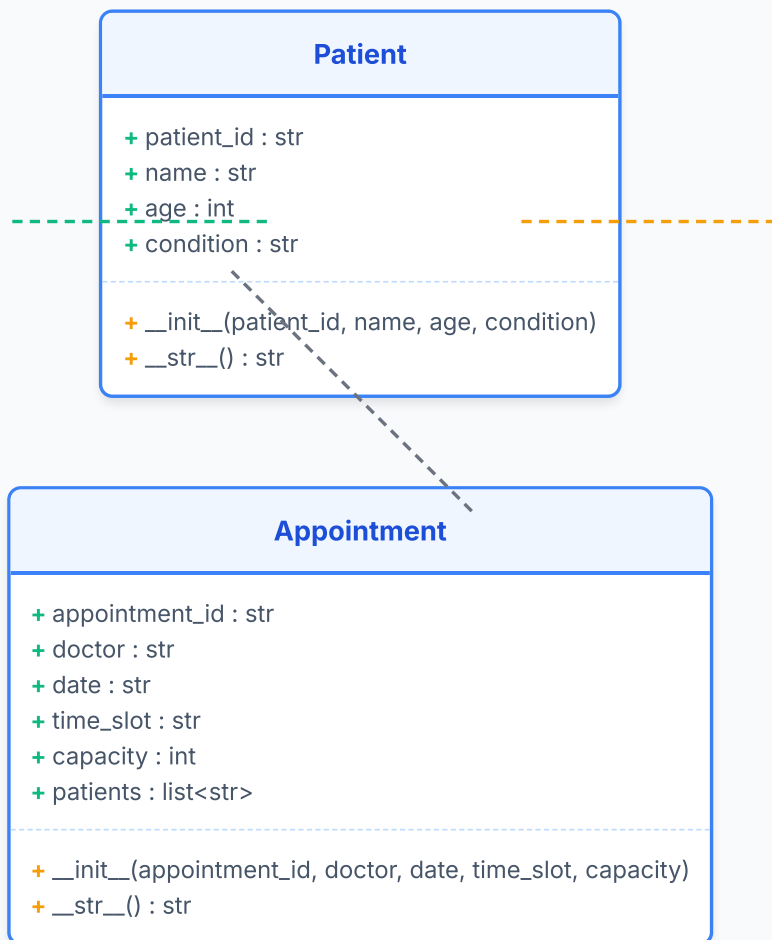
Advanced Concepts: Modular Design, Menu-Driven CLI, Report Generation.

Project File Structure

```
└─Project Root/  
  └─instructions.txt (Project Guidelines)
```

- └main.py (Entry Point & Menu)
- └appointment_manager.py (Core Logic & Data Handling)
- └patient.py (Patient Class)
- └appointment.py (Appointment Class)
- └exceptions.py (Custom Error Classes)
- └report.py (Report Functions)

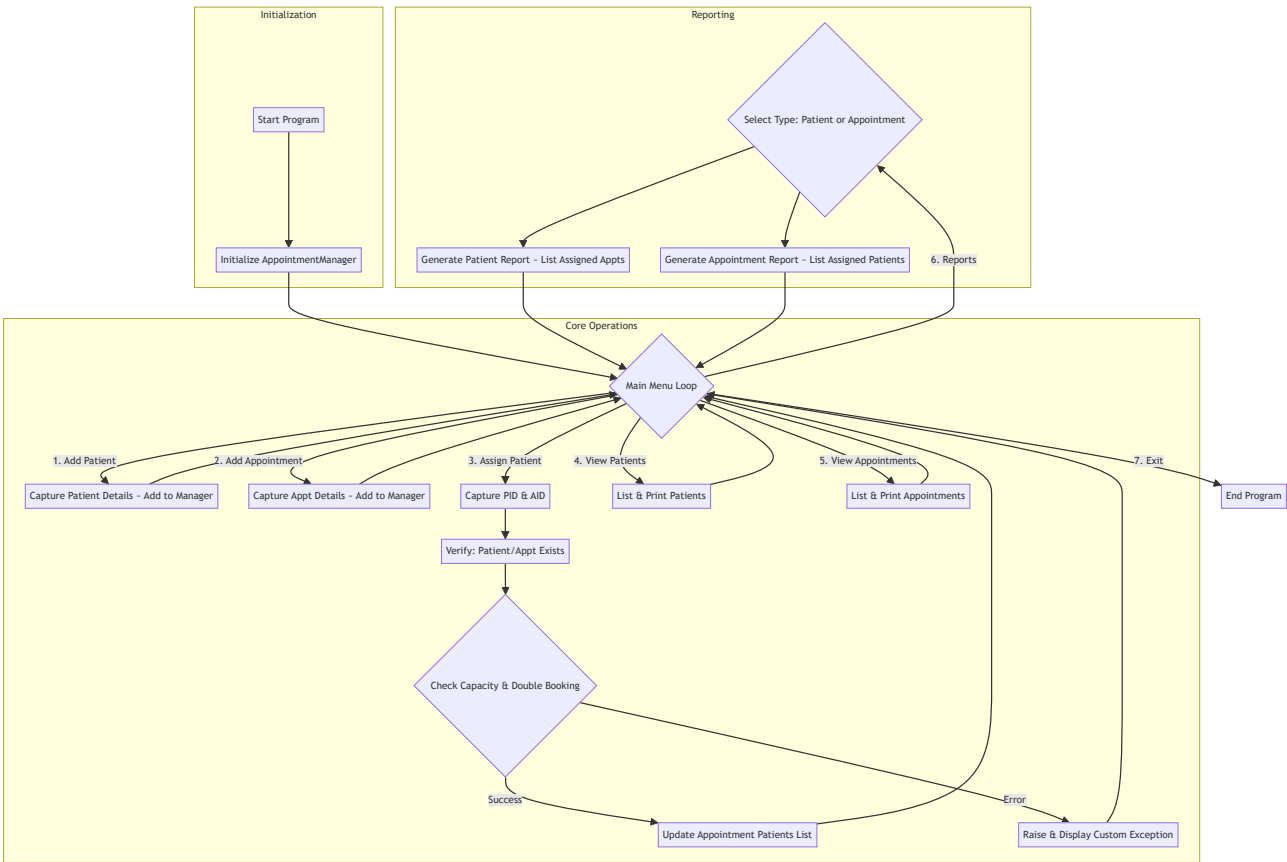
UML Class Diagram (Interactive Conceptual View)



Key Relationships & Dependencies:

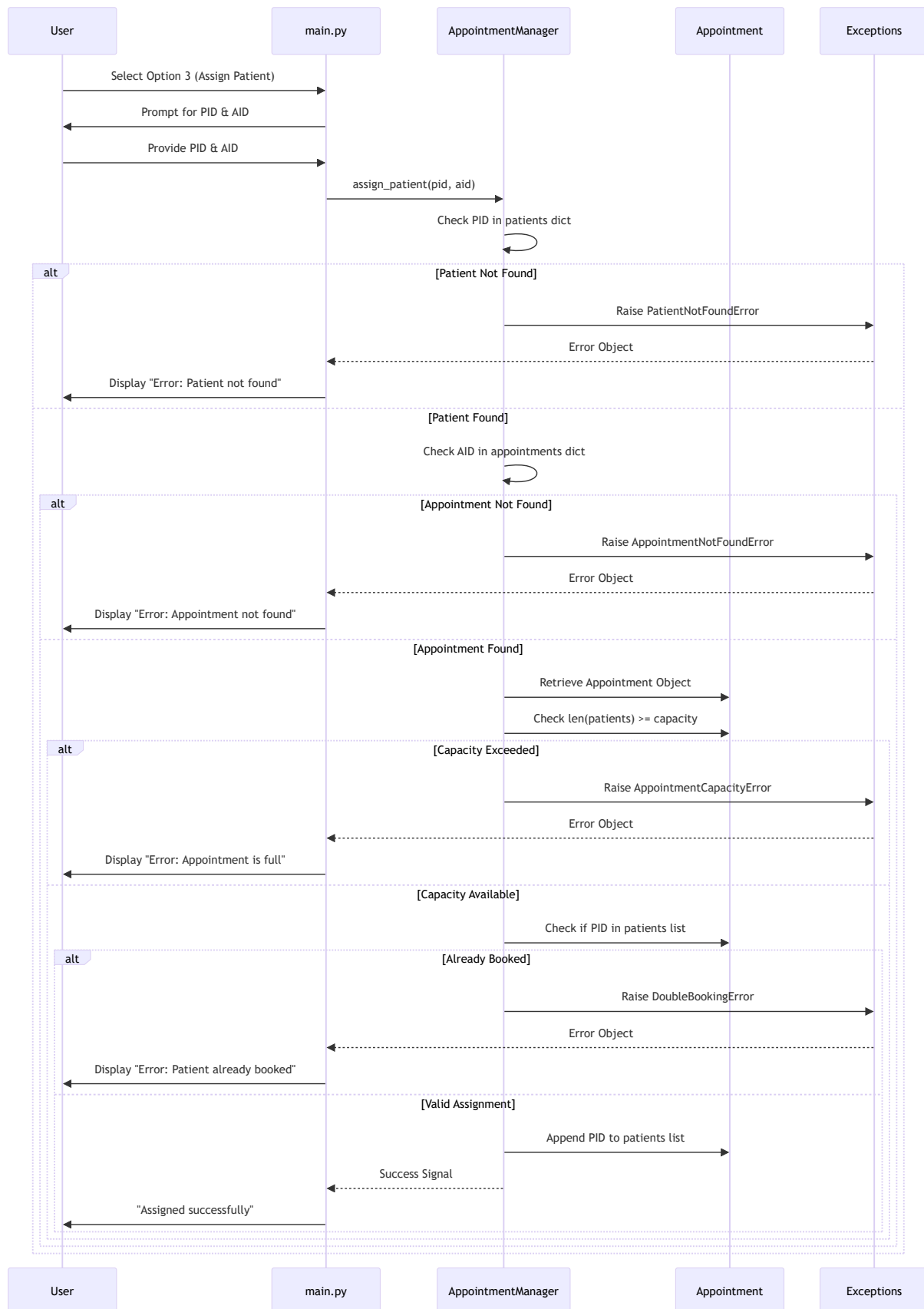
- **Composition (Green Line):** AppointmentManager manages multiple Patients and Appointments.
- **Association (Amber Line):** Appointments link to Patients via ID lists.
- **Dependency (Red Dotted Line):** Manager raises exceptions from exceptions.py.
- **Note:** Hover over classes for interactive zoom effect.

Application Workflow (Main Menu)



This enhanced flowchart organizes operations into subgraphs for better modularity visualization. It highlights user flows, validation checks, and reporting paths.

Sequence Diagram: Patient Assignment Process



This detailed sequence diagram maps the assignment flow, including all conditional branches and error messages. It illustrates interactions between components for a clear understanding of runtime behavior.

Detailed Module Reference

File: instructions.txt

```
# Content of instructions.txt
=====
Patient and Appointment Management
=====

Domain: Healthcare / Patient Scheduling
Problem: Prevent double-booking, overfilled slots, and lack of visibility.

Key Features:
- Create patients (ID, name, age, condition).
- Create appointments (doctor, time, capacity).
- Assign patients, ensuring no double-booking or capacity overflow.

Learning Goals:
- Object-Oriented Programming (Classes, Relationships)
- Data Structures (Dictionaries for lookups)
- Exception Handling (Custom Exceptions)
- Modular Code (Separate files for entities and logic)

Student Breakdown:
- Student A: Patient + Appointment classes.
- Student B: AppointmentManager + Exceptions.
- Student C: Reporting + Main menu (main.py).
```

Detailed Explanation: Serves as the foundational **Project Specification**. It defines the scope, features, and team roles, promoting collaborative development while aligning with educational goals like OOP and modularity.

File: main.py

```
# main.py
from patient import Patient
from appointment import Appointment
from appointment_manager import AppointmentManager
from report import patient_report, appointment_report

def main():
    manager = AppointmentManager()

    while True:
        print("\nPatient & Appointment Management")
        print("1. Add Patient")
        print("2. Add Appointment")
        print("3. Assign Patient to Appointment")
        print("4. View Patients")
        print("5. View Appointments")
        print("6. Reports")
        print("7. Exit")
        choice = input("Enter choice: ")

        if choice == "1":
```

```

pid = input("Patient ID: ")
name = input("Name: ")
age = int(input("Age: "))
cond = input("Condition: ")
manager.add_patient(Patient(pid, name, age, cond))
print("Patient added.")

elif choice == "2":
    aid = input("Appointment ID: ")
    doc = input("Doctor: ")
    date = input("Date (YYYY-MM-DD): ")
    slot = input("Time Slot (e.g. 09:00-09:30): ")
    cap = int(input("Capacity: "))
    manager.add_appointment(Appointment(aid, doc, date, slot, cap))
    print("Appointment added.")

elif choice == "3":
    pid = input("Patient ID: ")
    aid = input("Appointment ID: ")
    try:
        manager.assign_patient(pid, aid)
        print("Assigned successfully.")
    except Exception as e:
        print("Error:", e)

elif choice == "4":
    for p in manager.list_patients():
        print("-", p)

elif choice == "5":
    for a in manager.list_appointments():
        print("-", a)

elif choice == "6":
    sub = input("Report type (p=patient / a=appointment): ")
    if sub == "p":
        pid = input("Patient ID: ")
        print(patient_report(manager, pid))
    else:
        aid = input("Appointment ID: ")
        print(appointment_report(manager, aid))

elif choice == "7":
    print("Exiting...")
    break

else:
    print("Invalid choice.")

if __name__ == "__main__":
    main()

```

Detailed Explanation: The **Application Entry Point**. Initializes the manager, runs a persistent menu loop for user interactions, and handles errors globally. It integrates all modules into a cohesive CLI experience.

```

# appointment_manager.py
from exceptions import (
    DoubleBookingError,
    AppointmentCapacityError,
    PatientNotFoundError,
    AppointmentNotFoundError
)

class AppointmentManager:
    def __init__(self):
        self.patients = {}      # patient_id -> Patient object
        self.appointments = {}  # appointment_id -> Appointment object

    def add_patient(self, patient):
        self.patients[patient.patient_id] = patient

    def add_appointment(self, appointment):
        self.appointments[appointment.appointment_id] = appointment

    def assign_patient(self, patient_id, appointment_id):
        if patient_id not in self.patients:
            raise PatientNotFoundError("Patient not found")
        if appointment_id not in self.appointments:
            raise AppointmentNotFoundError("Appointment not found")

        appointment = self.appointments[appointment_id]

        if len(appointment.patients) >= appointment.capacity:
            raise AppointmentCapacityError("Appointment is full")

        if patient_id in appointment.patients:
            raise DoubleBookingError("Patient already booked in this appointment")

        appointment.patients.append(patient_id)

    def list_patients(self):
        return list(self.patients.values())

    def list_appointments(self):
        return list(self.appointments.values())

```

Detailed Explanation: The **Business Logic Core**. Manages data with efficient dictionaries, enforces rules via sequential checks in `assign_patient`, and raises targeted exceptions for robustness.

File: appointment.py

```

# appointment.py

class Appointment:

    def __init__(self, appointment_id, doctor, date, time_slot, capacity=1):
        self.appointment_id = appointment_id
        self.doctor = doctor
        self.date = date
        self.time_slot = time_slot
        self.capacity = capacity

```

```

        self.patients = [] # store patient IDs

    def __str__(self):
        return f"[{self.appointment_id}] Dr. {self.doctor}, {self.date} {self.time_slot}, {len(self.patients)} patients"

```

Detailed Explanation: Defines the **Appointment Entity**. Focuses on data storage and string representation; validation is delegated to the manager for better separation of concerns.

File: exceptions.py

```

# exceptions.py
class DoubleBookingError(Exception):
    pass

class AppointmentCapacityError(Exception):
    pass

class PatientNotFoundError(Exception):
    pass

class AppointmentNotFoundError(Exception):
    pass

```

Detailed Explanation: Custom **Error Handling Classes**. These extend Exception to provide semantic error types, enabling precise catching and user-friendly messages throughout the system.

File: patient.py

```

# patient.py

class Patient:
    def __init__(self, patient_id, name, age, condition=""):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.condition = condition

    def __str__(self):
        return f"{self.patient_id} - {self.name}, {self.age} yrs ({self.condition})"

```

Detailed Explanation: Defines the **Patient Entity**. A simple data class with initialization and string formatting for easy integration in views and reports.

File: report.py

```

# report.py

def patient_report(manager, patient_id):
    if patient_id not in manager.patients:

```



```

        return "Patient not found."
    patient = manager.patients[patient_id]
    lines = [f"Report for {patient.name} ({patient.patient_id}):"]
    for appt in manager.appointments.values():
        if patient_id in appt.patients:
            lines.append(f" - {appt}")
    return "\n".join(lines)

def appointment_report(manager, appointment_id):
    if appointment_id not in manager.appointments:
        return "Appointment not found."
    appt = manager.appointments[appointment_id]
    lines = [f"Report for {appt}"]
    for pid in appt.patients:
        p = manager.patients[pid]
        lines.append(f"    * {p.name}, {p.age} yrs")
    return "\n".join(lines)

```

Detailed Explanation: Handles **Report Generation**. Functions cross-reference manager data to produce formatted summaries, supporting patient-centric or appointment-centric views without altering state.